



CONFIDENTIAL — PRODUCT

Automation Platform Guide

Quenyx vOPS HUB — v1.0.0 RC1 · Document v1.0

Document Version	1.0
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Confidential — Product
Owner	Platform Engineering
Status	Released
Last Updated	2026-06-30
Document Type	Platform guide

REVISION HISTORY

Version	Date	Notes
1.0	2026-06-30	Initial Automation Platform guide (Sprint 23): registry-driven, safe-by-default execution shared across modules.

Table of Contents

- 1. Mental model
- 2. Components
- 3. Execution adapters (the contract)
- 4. The safety envelope (read this twice)
- 5. Adding a new execution runner
- 6. Automation Learning (no self-training)
- 7. Consuming the platform from a future module

24 — Automation Platform Guide

Audience: Platform engineers, SREs, and module developers. **Scope:** The **shared Automation Platform** — the reusable engine that future modules and AI consume. Automation is a **platform capability**, not logic isolated inside QynRun.

1. Mental model

```

Quenyx module / AI
  ↓
Action / Workflow / Runbook (data-only definitions)
  ↓
Execution Engine — resolves adapter via → Automation Registry
  ↓                                     (ssh, powershell, rest,
Approval gate (live only)                webhook, script, docker,
  ↓                                     kubernetes, oci, aws, azure, gcp ...)
Execution Adapter — the ONLY place a side effect can occur
  ↓
Execution record + steps → Audit → Automation Learning
  
```

There is exactly **one** place that performs a side effect (the adapter), exactly **one** engine that drives it, and **no** hardcoded execution path. New runners plug in by registering one adapter — mirroring the AI Adapter Platform.

2. Components

Component	Responsibility
Automation Registry (<code>AutomationAdapterRegistry</code>)	Dynamic registration + discovery of execution adapters.
Action Registry (<code>ActionRegistry</code>)	Catalog of reusable named actions (key → adapter + schema + destructive/rollback flags). Seeded from <code>config/automation.php</code> , extensible at runtime.
Execution Engine (<code>ExecutionEngine</code>)	The single execution path: resolves the adapter, enforces the safety envelope, persists executions + steps, audits, records learning.
Workflow Engine (<code>WorkflowEngine</code>)	Validates and runs workflow definitions (trigger → conditions → actions → approval → execution → verification → notification → audit).
Runbook Engine (<code>RunbookEngine</code>)	Runbook validation + execution. AI-assisted runbooks are editable drafts and are never auto-executed.
Approval Engine (<code>ApprovalEngine</code>)	The human gate in front of every live action.
Rollback Engine (<code>RollbackEngine</code>)	Adapter-driven undo of a successful live execution.
Execution History (<code>ExecutionHistory</code>)	Read model over executions + steps.
Automation Learning (<code>AutomationLearningService</code>)	Immutable, auditable outcome records; aggregated stats the AI cites.
Audit (<code>AutomationAuditLogger</code>)	WHO requested/approved/executed/rolled-back WHAT and the outcome.

3. Execution adapters (the contract)

Every adapter implements `App\Contracts\Automation\ExecutionAdapter` :

- `key/name/description/category/capabilities`
- `supportsRollback()` / `isOperational()`
- `parameterSchema()` — declared fields for UI + validation
- `execute(ExecutionContext): ExecutionResult`
- `rollback(ExecutionContext, $token): ExecutionResult`

Safe by default. Adapters MUST honor `ExecutionContext::isDryRun()` : in dry-run they return a deterministic **plan** and perform no side effect. `ExecutionResult` statuses are `dry_run`, `succeeded`, `failed`, `skipped` (live requested but no runner — honest, no side effect).

Shipped adapters: `ssh`, `powershell`, `rest`, `webhook`, `script`, plus registry-discoverable **planned** runners `docker`, `kubernetes`, `oci`, `aws`, `azure`, `gcp` (real runners swap in later with no engine/API change).

4. The safety envelope (read this twice)

1. **Master switch** `automation.live_execution` (env `AUTOMATION_LIVE_EXECUTION`) is **OFF by default** → every action is a dry-run plan.
2. **Approval gate** — even with live enabled, every live action is created as `awaiting_approval` and runs only after an authorized operator (`administerAi`) approves it. No automatic destructive action is possible.
3. **HTTP allowlist** — REST/Webhook only call hosts on `automation.http.allowed_hosts` .
4. **Runner flags** — `script / ssh / powershell` live execution each require their own `*_runner_enabled` flag; otherwise live requests return `skipped` (honest).
5. **Rollback** — only a `succeeded` live execution on a rollback-capable adapter can be rolled back.
6. Everything is **workspace-scoped, UUID-only, RBAC-gated, audited**.

5. Adding a new execution runner

1. Implement `ExecutionAdapter` (extend `AbstractExecutionAdapter`).
2. Register it in `AppServiceProvider::boot()` (`$automation→register( ... )`).
3. Optionally add actions to `config/automation.php` .

No engine, registry, controller, or route changes. That is the whole point.

6. Automation Learning (no self-training)

Each execution writes an immutable `AutomationLearningRecord` (outcome, duration, adapter, mode; operator feedback optional). `stats()` aggregates success/failure/rollback rates and typical duration per action. The AI **cites** these aggregates — there is **no model training, no hidden state**, and every record is inspectable and workspace-scoped.

7. Consuming the platform from a future module

A module never re-implements execution. It builds a definition (action/workflow/runbook) and calls the engine (directly or via the QynRun APIs). Cross-module flows reuse the **AI Adapter Registry** (see the AI Adapter Developer Guide) for context — never by branching on module names.